

Conflict-guided Assignment Search for Multi-agent Task Assignment and Pathfinding

Sushrut Surve, Jovan Menezes, Silvia Ferrari
Sibley School of Mechanical and Aerospace Engineering,
Cornell University,
Ithaca, NY 14850, USA
{ss4248, jcm483, ferrari}@cornell.edu

Abstract—Multi-agent teaming often involves assigning tasks to agents and finding collision-free paths to achieve them, also known as the task assignment and pathfinding (TAPF) problem. Existing approaches often explore task assignments randomly and rely on computationally intensive methods to generate candidate solutions, resulting in long computation times and low success rates as the problem scales. This paper introduces Conflict-Based Search with Swapping (CBS-Swap), a novel approach that systematically explores task assignments by leveraging collisions detected in candidate paths. By strategically swapping tasks between agents involved in conflicts, CBS-Swap efficiently generates assignment candidates that directly target conflict resolution, with minimal computational overhead. To validate the approach, a cyber-physical warehouse automation testbed is developed for hardware-in-the-loop (HIL) testing of TAPF algorithms in realistic environments. The effectiveness of CBS-Swap is demonstrated through both simulation and experiments on the physical testbed.

I. INTRODUCTION

Multi-agent robotic systems are useful in complex and highly collaborative tasks such as search and rescue [1], [2], collaborative assembly [3], and warehouse automation [4], [5]. In all these application scenarios, a key desired ability from the team of robots is to collaborate to allocate tasks among themselves and plan collision-free paths. This problem, known as task assignment and pathfinding (TAPF) [6], becomes increasingly significant in environments such as automated warehouse where congestion, resulting from narrow corridors or a large number of agents operating simultaneously, needs to be tackled by planning efficiently. However, TAPF problems are NP-hard [7], and it is important that solution algorithms scale with the number of agents and provide solutions fast enough to be used in real-time applications.

The TAPF problem is also referred to as unlabelled multi-agent pathfinding (MAPF) problem [8], where labeling involves assigning tasks to each robot. This paper focuses on centralized solutions for the TAPF problem. A simple but suboptimal solution to the unlabeled MAPF problem is to first solve for the assignment and convert the problem to a labeled MAPF problem, which can be solved by optimal algorithms such as Conflict Based Search (CBS) [9]. Several works extend CBS to find optimal solutions to the TAPF problem. Conflict Based Search with Task assignment (CBS-TA) is one such popular approach that leverages the search

strategy of CBS to span the assignment and path planning space completely and eventually find an optimal solution [8].

Although CBS-TA guarantees optimal solutions, it is computationally expensive, resulting in scalability challenges that limit its real-world applicability [7]. One of the primary contributors to this issue is the computationally expensive assignment problem that needs to be solved very frequently. Moreover, the strategy used to augment the assignment problem to generate assignment candidates is random. When solving the assignment is expensive, this randomness can lead to a high time-to-solution. An incremental target assignment approach was developed to tackle the computational complexity issue, but the resulting algorithm is not guaranteed to be complete [7]. The key intuition of this paper is that conflicts between multi-agent paths can be used to guide the assignment candidate sampling by generating new candidates by swapping the assignments of agents in conflict. Thus, the conflicts inferred during CBS-TA can be exploited not only for pathfinding but also to inform task assignment sampling, thereby enhancing conflict resolution efficiency. When used as the primary approach for generating candidates and supplemented by solving the assignment problem only when necessary, this strategy significantly reduces computational complexity while ensuring completeness and enhancing performance.

This paper presents an optimal and complete algorithm, named Conflict-Based Search with Swapping (CBS-Swap), to solve the TAPF problem by using conflicts to guide the exploration of the assignment space. A novel swapping strategy, presented in Section IV, is used to incorporate this conflict information into the assignment generation process. The computational complexity of the algorithm is analyzed in Section V-A. Section V-B provides theorems to prove the optimality and completeness of CBS-Swap. To test this framework in a warehouse automation setting with robot hardware, a new hardware-in-the-loop (HIL) cyber-physical testbed, presented in Section IV-D is developed which interfaces real and simulated robots using photorealistic virtual warehouse environments. Finally, this approach is tested in simulation and in the cyber-physical testbed, the results from which are presented in Section VI.

II. RELATED WORK

A. Multi-agent pathfinding

MAPF is the problem of planning collision-free paths to designated goal positions in a shared environment while optimizing an objective. The two most commonly used objectives are the total travel time of the multi-agent team and the maximum individual travel time, also called makespan [10]. The MAPF problem optimizing these objectives is NP-hard to solve optimally [11], [12]. This paper focuses on centralized approaches for solving the MAPF problem. The centralized approaches can be broadly categorized into sub-optimal and optimal approaches. Sub-optimal approaches aim to trade optimality for computational complexity to achieve time-efficient solutions [13]–[15]. Optimal approaches focus on preserving the optimality of the solutions by conducting a global search [16]–[18]. CBS is a widely adopted approach, valued for its efficiency while maintaining both optimality and completeness. Its efficient two-level search framework has since been adopted by numerous algorithms in the literature. [8], [19], [20].

B. Task Assignment

Task assignment is a classical combinatorial optimization problem that involves assigning tasks to agents such that each task is allocated to at most one agent, with the objective of minimizing the total assignment cost. When the total cost of an assignment is the sum of the cost of each agent-task pair in the assignment, the problem is referred to as a linear assignment problem. Several exact [21]–[23] and approximate [24] solutions have been proposed in the literature. However, most popular approaches exhibit high computational complexity [21].

C. Simultaneous Task Assignment and Pathfinding

MAPF can be viewed as a special instance of the TAPF where the goal assignments are known and hence the TAPF problem also NP-hard [7]. A naive but intuitive solution to the problem is solving for the assignment and then use MAPF solution algorithms to get collision-free paths. However, this sequential approach relies on the assumption that task assignment and pathfinding can be treated independently — an assumption that is generally invalid [25]. Thus, several works have explored ways to solve for the assignment and multi-agent paths simultaneously [6], [8], [25]. These approaches however are either not applicable for optimizing total travel time or are not applicable to scenarios with unequal number of agents and task locations. CBS-TA is a popular complete and optimal approach to solve the TAPF problem while minimizing total travel time [8]. CBS-TA extends the two-level search framework of CBS to construct a search forest that simultaneously addresses goal assignment and multi-agent path planning. However, CBS-TA incurs high computational complexity owing to the target assignment problem that needs to be solved at every iteration of the algorithm and continues to use path-finding as a conflict resolution only, as in CBS, and not in assignment search. Incremental target assignment CBS has been proposed to

reduce the computational complexity of TAPF algorithms [7]. However, it does not guarantee completeness and still relies on pathfinding for conflict resolution.

D. Swapping for conflict resolution

Swapping agent positions has been used as a conflict resolution strategy in solution approaches for the MAPF [13] and the multi-agent pickup and delivery problem [26]. Recently, TSWAP introduced swapping task assignments alongside swapping agent positions to resolve conflicts encountered while solving the TAPF problem [4], [27]. This algorithm however is suboptimal.

III. PROBLEM FORMULATION

This paper considers the problem of optimizing the task assignment and path traversals of a multi-robot team tasked at reaching the task locations in minimum time. The robots and tasks are assumed to be distributed across a discretized workspace, represented as an undirected graph $G = (V, E)$, where each vertex $v \in V$ represents a location, and each edge $e \in E$ denotes a direct passage between two locations with a unit cost. This workspace is populated with stationary objects, acting as obstacles for robot motion, and the set of vertices occupying these obstacles, denoted by $\mathcal{O} \subset V$ is assumed known.

The multi-robot team comprises n agents, indexed by $\mathcal{I} \triangleq \{1, 2, \dots, n\}$ each starting at a unique location $s_i \in V \setminus \mathcal{O}, i \in \mathcal{I}$. Additionally, there exists a set of m tasks distributed across the workspace at known locations, $t_j \in V \setminus \mathcal{O}, j \in \mathcal{J}$ where $\mathcal{J} \triangleq \{1, \dots, m\}$. Robots are assigned task locations to reach, which is defined compactly by a binary assignment matrix A of size $n \times m$, where, $A_{(i,j)}$ is 1 if the agent i is assigned to task j and 0 otherwise. It is assumed that A belongs to the space $\mathcal{A}$ of task assignments in which a task is assigned to at most one robot and a robot is assigned to at most one task.

At each discrete time step, an agent may either move to an adjacent vertex (traversing an edge) or remain stationary at its current location. Let $v_i(k)$ denote the position of agent i at time step k , then the path P_i is defined as the set of vertices agent i traversed until final time T_i , denoted by $P_i \triangleq [v_i(0), v_i(1), \dots, v_i(T_i)]$. The path P_i is constrained to belong to the space $\mathcal{P}(\mathcal{A})$, which denotes the family of all feasible paths that satisfy the following constraints:

- 1) The agent starts at its designated start location:
 $v_i(0) = s_i$.
- 2) The agent reaches and remains at any of task locations:
 $v_i(k) = t_j, \forall k \geq T_i$ such that $A_{(i,j)} = 1, A \in \mathcal{A}$.
- 3) The movement of the agent is constrained to adjacent vertices or waiting at the same vertex:
 $\forall k \in \{0, \dots, T_i\}, [v_i(k), v_i(k+1)] \in E$ or $v_i(k) = v_i(k+1)$.
- 4) If no specific task is assigned, agent i remains at s_i .

The set of feasible paths for all agents in the multi-robot team, denoted by $S \triangleq \{P_i\}_{i=1}^n$, is termed as a *solution*. A *conflict* arises between two agents i and i' in a solution, denoted by a tuple (i, i', v, k) , when agent i and i' occupy

the same vertex $v \in V$ at time step k . A solution in which there are no conflicts between any agents is called a *valid* solution. It is assumed that agents move in discrete time steps of equal duration, and hence the cost of traversing a path is equal to the final time T_i . The objective is to find an optimal valid solution that minimizes the sum of individual path costs (SIC), which is summarized mathematically as follows:

$$S^* = \arg \min_{S \in \mathcal{P}(\mathcal{A})} \sum_{i=1}^n T_i. \quad (1)$$

This problem involves searching for both task assignments and conflict-free paths, which is proven to be NP-hard [28]. Finding a solution would require efficiently searching the joint space $\mathcal{P} \times \mathcal{A}$. Existing methods do not conduct this search efficiently, leading to lower success rates and longer times to find solutions. Therefore, the next section develops a novel method to search the assignment space and path planning space efficiently by leveraging conflict detection.

IV. METHODOLOGY

CBS-Swap, similar to CBS, comprises a two-level search, namely, the high level and the low level. The high-level search is responsible for evaluating the current candidate set of paths for conflicts and generating new candidate assignments upon detection of collisions. The low-level uses these detected collisions to plan new paths that avoid them. The CBS-Swap algorithm framework is summarized in Figure. 1.

A. Terminology

A constraint (i, v, k) is a tuple that specifies that the agent i is prohibited from being at vertex v at time k . CBS-Swap builds a constraint forest which is a set of constraint trees (CTs), where each tree is associated with a candidate assignment A . Every node N in this CT consists of four parameters, namely, the assignment $N.A$, constraint set $N.constraints$, solution $N.S$, and the SIC associated with the solution $N.cost$. The child of a node in a CT inherits the constraints of the parent node and adds one new constraint for one agent. A *consistent* path P_i satisfies all constraints in the constraint set pertaining to agent i and consequently, the set of paths that satisfy all constraints for all agents is called a consistent solution. Note that a consistent solution can be invalid if there are conflicts between agent paths. The goal is to find a valid and consistent solution.

The high level is responsible for selecting a candidate node to be evaluated, detecting collisions in candidate solutions, and spawning new constraint trees. Additionally, the high level also generates constraint sets from detected collisions, which are added to the nodes, thus making them inconsistent. The lower level is responsible for spawning new nodes in the same constraint tree with consistent solutions, using the constraint set generated by the high level. This two-level search process is iterated until the high level finds a valid and consistent solution which is then declared as the goal node.

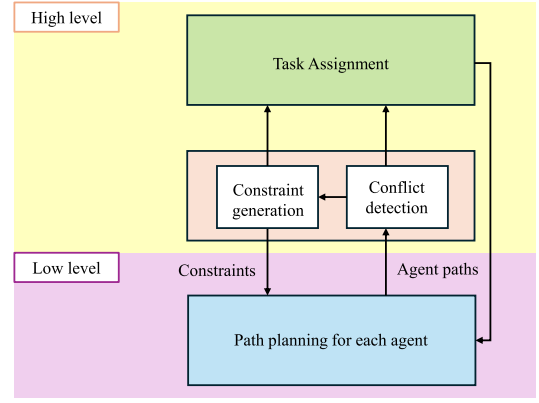


Fig. 1. CBS-Swap comprises a high-level search that detects conflicts in candidate solutions and generates new task assignments, and a low-level search that modifies paths in existing solutions to resolve those conflicts.

B. High-level

In the high-level search, a candidate node is selected using best-first search over the constraint forest, where nodes are prioritized based on their associated costs. Conflicts in the selected node's solution are then identified and used to generate new constraints. These constraints are used to expand existing constraint trees and to spawn new ones using a novel conflict-guided assignment strategy. The high level search is summarized in Algorithm.1. The next subsections describe the constraint generation process and the conflict-guided assignment strategy.

1) *Collision detection and constraint generation:* Once a node N is selected, the solution is evaluated to check for conflict or collision between paths, as defined by the conditions in the section. III. For every time step k , it is checked if the paths for each pair of agents (i, i') satisfy the condition for conflict. All the conflicts that satisfy this condition are stored as a tuple (i, i', v, k) in a set of conflicts *conflicts*. Every conflict can be used to generate two constraints, (i, v, k) and (i', v, k) respectively. Thus, every node with r independent conflicts (i.e. $(i, i', v, k) = (i', i, v, k)$) will lead to a maximum of $2r$ new constraints, and hence a maximum of $2r$ new nodes in the constraint tree. A set of constraints, $N.constraints$, is built with these new constraints. The resulting constraint set, constructed as a result of this process, is used by the lower level to spawn new nodes and used by the novel task assignment strategy presented in this paper.

2) *Conflict-guided assignment sampling:* The sampling approach presented in this section aims to generate new assignment candidates for spawning new trees in the constraint tree. In this process, the search expands multi-dimensionally, exploring new assignment strategies alongside new paths generated by the lower level for the existing assignment candidates. Previous work [8] has presented a constrained assignment algorithm to generate new assignment candidates. This method generates assignment candidates by taking as input the assignments that need to be kept, the assignments that need to be removed, and the cost matrix C which quantifies the cost of assigning each agent to a task. With

Algorithm 1 high-level of CBS-Swap

Input: Graph, start and task locations, assignment matrix
Result: optimal path for each agent

```

1:  $R.constraints \leftarrow \emptyset$ 
2:  $R.assignment \leftarrow firstAssignment()$ 
3:  $R.root \leftarrow \mathbf{True}$ 
4:  $R.S \leftarrow \text{find individual paths using low-level}()$ 
5:  $R.cost \leftarrow SIC(R.S)$ 
6: insert  $R$  to OPEN
7: while OPEN not empty do
8:    $P \leftarrow \text{best node from OPEN // lowest solution cost}$ 
9:   Detect conflicts in  $P$  and record them in  $conflicts$ .
10:  if  $P$  has no conflict then
11:    return  $P.S$  //
12:  end if
13:   $current\_assignment \leftarrow P.assignment$ 
14:  if  $P.root$  is True then
15:     $R \leftarrow \text{new node}$ 
16:     $R.constraints \leftarrow \emptyset$ 
17:     $R.assignment \leftarrow nextAssignment()$ 
18:     $R.root \leftarrow \mathbf{True}$ 
19:     $R.S \leftarrow \text{find individual paths using low-level}()$ 
20:     $R.cost \leftarrow SIC(R.S)$ 
21:    insert  $R$  to OPEN
22:  end if
23:   $(i, i', v, k) \leftarrow \text{first conflict in } P$ 
24:  for agent  $i$  in Conflict do
25:     $Q \leftarrow \text{new node}$ 
26:     $Q.constraints \leftarrow P.constraints + (i, v, k)$ 
27:     $Q.assignment \leftarrow P.assignment$ 
28:     $Q.root \leftarrow \mathbf{False}$ 
29:     $Q.S \leftarrow P.S$ 
30:    Update  $Q.S$  by invoking low-level
31:     $Q.cost \leftarrow SIC(Q.S)$ 
32:    Insert  $Q$  to OPEN
33:  end for
34: end while

```

distance as the cost metric, the cost matrix is populated by using planning algorithms to find the shortest path between each agent and task pair. For agents that cannot reach a specific task location due to the map structure, the cost is set to an arbitrarily high value. Consequently, the Hungarian algorithm is used to solve the linear assignment problem by setting the cost of assignments that should be preserved to zero and those that should be removed to an arbitrarily high value [21]. The assignment for the first node in the search forest is computed by the function *firstAssignment* in Algorithm 1, which also applies the Hungarian algorithm to the original, unmodified cost matrix [8]. A key limitation of this candidate sampling approach is that the selection and removal of assignments are arbitrary, with current implementations relying on lexicographic ordering to ensure completeness.

This paper introduces a novel heuristic strategy to generate assignment candidates by leveraging the conflict sets already

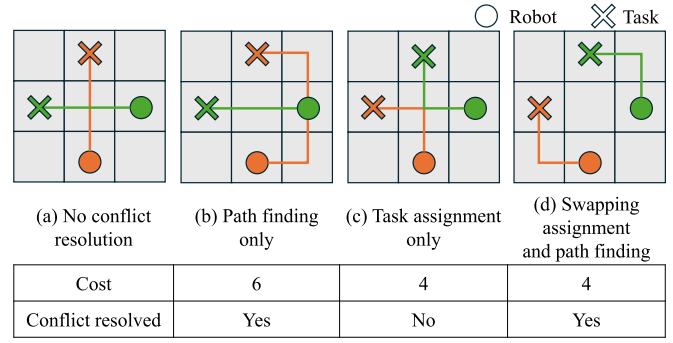


Fig. 2. Example to show that swapping assignments of conflicting agents with pathfinding can be used as a conflict-resolution strategy.

constructed by the high level to inform the lower level. For every conflict (i, i', v, k) , the new assignment is generated by swapping the tasks assigned to i and i' in the current constraint tree. This development is motivated by the fact that certain task assignments obtained from swapping may result in fewer conflicts by directly trying to resolve them, as shown with an example in Figure 2. Thus, while the lower level tries to search the space of feasible paths $\mathcal{P}$ using conflicts, this assignment sampling strategy ensures that the high level also explores the task assignment space $\mathcal{A}$ based on these conflicts. Thus, for every node with r conflicts, r new constraint trees are spawned with the constraint set initialized as a null set. The advantage of this strategy is that it introduces assignment candidates specifically aimed at resolving conflicts with minimal computation (Section V-A) while significantly improving performance (Section VI). A simple example, presented in Figure 2, provides intuition for why this strategy is effective. This swapping strategy augments the constrained assignment algorithm to increase efficiency of assignment search while ensuring it is complete. The resulting hybrid search approach is summarized in Algorithm. 2 under the function name *nextAssignment*.

C. Low-level

For every element in the constraint set, a new node is spawned from the current candidate node, which inherits the constraints from the parent node (current candidate node), and with this element added to its constraint set. This new constraint renders the previous solution, inherited from its parent, inconsistent. The goal of the lower level is to use this new constraint set and the assignment set inherited from its parent to generate a new consistent solution for the new node. A path planning algorithm, such as A* [29], is used to construct a path from the starting location of each agent to its assigned task location while abiding by the new constraint set. The new nodes, formed as a result, are added to the constraint tree and consequently to the constraint forest. In further iterations of the high-level, these nodes are considered for candidate node selection conducted by the high-level.

D. Cyber-Physical System for Warehouse Automation

To demonstrate the practicality of the presented algorithm, CBS-Swap is implemented on ground robots in real-

Algorithm 2 nextAssignment

Input: current_solution, cost matrix C , ASG_OPEN**Result:** next best assignment, updated ASG_OPEN

```
1: for  $i \leftarrow 1$  to  $n$  do
2:   for  $\text{conflict}$  in  $P.\text{conflict}$  do
3:      $(i, i', v, k) \leftarrow \text{conflict}$ 
4:      $Q \leftarrow \text{new node}$ 
5:      $Q.S \leftarrow P.S$ 
6:      $Q.S[i] \leftarrow P.S[i']$ 
7:      $Q.S[i'] \leftarrow P.S[i]$ 
8:     if  $Q.S$  not empty then
9:       Insert  $Q$  to ASG_OPEN_SWAP
10:    end if
11:  end for
12: end for
13: if ASG_OPEN_SWAP not empty then
14:   Insert ASG_OPEN_SWAP to ASG_OPEN
15: else // no new assignment candidate from swapping
16:    $P \leftarrow \text{best node from ASG\_OPEN}$ 
17:   if  $P$  does not exist then
18:     return No next assignment
19:   end if
20:   for  $i \leftarrow 1$  to  $n$  do
21:     if  $i$  not part of  $P.I$  then
22:        $Q \leftarrow \text{new node}$ 
23:        $Q.S \leftarrow \text{constrainedAssignment}(C)$ 
24:       if  $Q.S$  not empty then
25:         Insert  $Q$  to ASG_OPEN
26:       end if
27:     end if
28:   end for
29: end if
30: return solution of best node from ASG_OPEN
```

world TAPF scenario. To bridge the gap between controlled laboratory test environments and real-world warehouse settings—where such multi-agent systems are commonly utilized—Real-Time Human-Autonomous Systems Collaboration (RealTHASC) [30] is utilized as a foundation. RealTHASC offers a cyber-physical testbed for developing multi-agent systems, incorporating simulated virtual robots, real robots, and real robots with digital twins within photorealistic environments, as shown in [31]. Leveraging this facility, CBS-Swap is evaluated in a photorealistic warehouse environment, which would be challenging to replicate in a traditional laboratory setting. Expanding on RealTHASC, a cyber-physical testbed is developed to enable collaboration between real and simulated robots within photorealistic virtual environments created in Unreal Engine (UE™). The base warehouse environment used to develop the testbed is obtained from the UE™ Marketplace. This environment is then modified to support virtual agents and digital avatars. The testbed also integrates a diverse range of sensors for perception, facilitating obstacle avoidance and real-time, perception-driven interaction among agents. This paper introduces a task allocation framework integrated

with RealTHASC, utilizing real-time data on robot and environment states to generate plans for both centralized and decentralized networks of agents, which may consist of a homogeneous or heterogeneous team of robots. To enhance efficiency, parallelization is employed, allowing lower-level motion planners and controllers for each agent to operate independently. Additionally, to accommodate larger virtual environments, a new workflow is developed for real-time motion-capture processing, incorporating scaling before and after the task planner without compromising data streaming efficiency. Figure 3 provides an overview of the warehouse cyber-physical system.

A warehouse environment is designed in UE™, tailored to the desired level of complexity. This virtual environment comprises multiple autonomous material handling vehicles (AMHVs) acting as agents commonly used in modern warehouses. These agents are either purely virtual entities operating within the digital environment or real-world agents with digital counterparts, whose movements are synchronized with real robots in the physical workspace via the motion capture system. Agents perceive their surroundings and detect each other in real-time using sensors mounted on individual agents within the UE™ environment. UE™ is interfaced with Python through the User Datagram Protocol to collect agent state data, task details, and environment state, which are then structured into a standardized data format. In MATLAB, the structured data is processed using the CBS-Swap algorithm which allocates tasks and generates paths for the multi-agent team to move towards their designated goals. Each agent's trajectory, computed by CBS-Swap, consists of waypoints representing the centers of the grid, guiding the agent from its start location to its assigned task location. These waypoints are then mapped to the world reference frame, and the resulting trajectory is transmitted to each agent's onboard low-level controller. The controller employs feedback linearization to ensure the trajectory adheres to the dynamics of the AMHVs, subsequently computing and executing the necessary linear and angular velocity commands. In this framework, control for virtual agents in UE™ is implemented in MATLAB, while real agents utilize a controller integrated within their ROS architecture. As virtual agents move based on computed velocity commands, real agents in the physical workspace are mirrored by their digital counterparts through the motion capture system, ensuring seamless synchronization between the real and virtual environments.

V. ANALYSIS

A. Computational complexity

The overall computational complexity of CBS-Swap is derived by analyzing the high-level and low-level separately. Given an optimal solution S^* , let $T_{\max}^* = \max_{i \in \mathcal{I}} \{T_i\}$ be the latest time that a single-agent's path $P_i \in S^*$ terminates in. The number of vertices in the grid is denoted by $|V|$. Thus, $nT_{\max}^*$ is the upper bound on the cost of an optimal solution for an upper bound on the cost of an optimal solution for the sum-of-costs objective. When multiple conflicts are

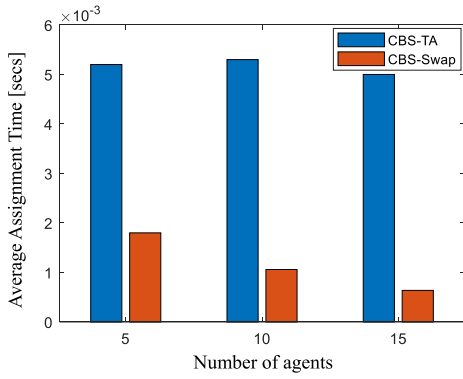


Fig. 5. CBS-Swap demonstrates significantly lower average assignment time as compared to CBS-TA.

average cost, and average runtime, with results averaged over 100 randomly generated instances for each agent count. The simulation results demonstrating the average success rate and average SIC over 100 runs are summarized in Figure. 6. CBS-Swap maintains a success rate above 80% for up to 10 agents, while CBS-TA drops below 60% beyond 6 agents. For 14 or more agents, CBS-Swap continues to achieve a success rate of 40–50%, whereas CBS-TA falls below 20%. Despite this substantial improvement in success rate, CBS-Swap maintains an average cost comparable to CBS-TA, as the cost curves remain closely aligned with only minor deviations. This indicates that CBS-Swap improves feasibility without significantly increasing computational cost. Consequently, CBS-Swap exhibits better scalability than CBS-TA, thus making it more suitable for congested environments such as large-scale industrial warehouses.

The average assignment time required to calculate the next best assignment candidate is estimated over these 100 random maps, and the values for 5, 10, and 15 agents are reported in the Figure. 5. It can be observed that the assignment time required by CBS-Swap is approximately an order of magnitude lower than the time required by CBS-TA. As stated in Section. V-A, this improvement can be primarily attributed to the swapping strategy, which avoids solving the computationally expensive assignment problem.

B. Physical experiments

An experiment is conducted using the framework outlined in Figure 3 within the cyber-physical testbed, consisting of two virtual agents, one real-world agent with a digital counterpart, and three tasks. Figure 7 (a) depicts the workspace used in the laboratory, while Figure 7 (b) presents a bird’s-eye view of the warehouse virtual environment developed in UETM. The dimensions of the virtual warehouse are 40.5m × 26.2m, whereas the laboratory workspace measures 3.35m × 2.1m. To enable task assignment using the CBS-Swap algorithm in MATLAB, the virtual environment is discretized into a grid of 14 rows and 21 columns, as illustrated in Figure 7 (c). The occupied cells are depicted in white, while the free cells are represented in grey shading. The discretization resolution is chosen so that each grid cell approximately corresponds to the size of a single forklift, allowing CBS-

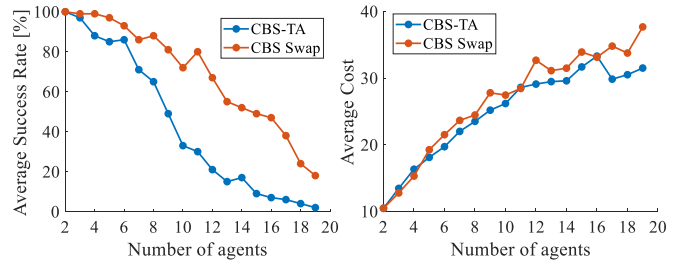


Fig. 6. Comparison of average success rate and cost over 100 random maps demonstrated by CBS-Swap with CBS-TA.

Swap to implicitly account for agent geometry. Agent start positions and task locations within the virtual environment are randomly selected across the warehouse space, as shown in Figure 7.

The CBS-Swap algorithm processes the agent start locations, task locations, and the discretized environment map as inputs to determine optimal task assignments and generate individual agent trajectories. The algorithm ensures that each agent is assigned a task while considering constraints such as obstacle avoidance and efficient path planning within the grid-based environment. For the specified start and task locations, the computed task assignments are as follows: Agent 01 is assigned to Task 03, Agent 02 is assigned to Task 01, and Agent 03 is assigned to Task 02. Once the assignments and conflict-free paths for each agent are determined, CBS-Swap provides the trajectories a sequence of waypoints that guide them from their initial positions to their designated task locations. These waypoints account for the agents’ geometry and ensure smooth navigation within the discretized map. Figure 7 (c) illustrates the assigned tasks along with the corresponding agent trajectories, visually representing the planned paths in the virtual warehouse environment.

VII. CONCLUSION

This paper presents CBS-Swap, a complete and optimal solution to the multi-agent TAPF problem that leverages conflicts between the paths to search the assignment space. Unlike previous TAPF algorithms, this algorithm methodically guides the search of the joint space of assignment and paths, by swapping the assignments of agents in conflict to generate new assignment candidates. This algorithm is shown to be more computationally efficient and demonstrates higher success rates in finding solutions than the baseline, making it more suitable to be used in real-world applications. Additionally, a cyber-physical testbed that interfaces real and virtual robots in photorealistic simulation environments is also presented and used to demonstrate the applicability of CBS-Swap in an automated (industrial) warehouse. Future work will provide solution feasibility guarantees, extend this algorithm to multi-task allocation (one robot can be assigned with more than one task), and explore human intervention with CBS-Swap for human-robot teaming.

REFERENCES

- [1] J. Banfi, J. Guzzi, A. Giusti, L. Gambardella, and G. A. Di Caro, “Fair multi-target tracking in cooperative multi-robot systems,” in *2015*

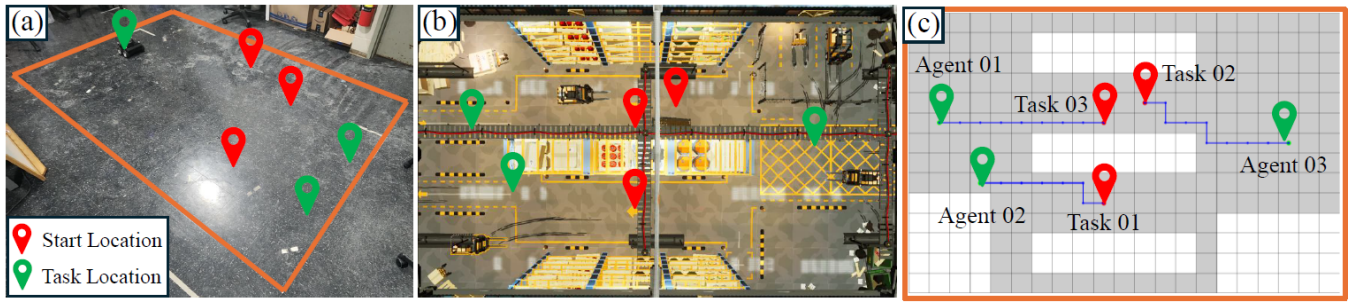


Fig. 7. Experimental setup for evaluating CBS-Swap in the cyber-physical system. (a) Real-world laboratory workspace featuring the physical robot along with designated start and goal locations for each agent. (b) Bird's-eye view of the virtual warehouse environment, highlighting the start and goal positions of AMHVs. (c) Discretized map in MATLAB utilized by CBS-Swap, showing start and goal locations along with the computed trajectories.

- IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2015, pp. 5411–5418.
- [2] J. Capitan, L. Merino, and A. Ollero, “Cooperative decision-making under uncertainties for multi-target surveillance with multiples uavs,” *Journal of Intelligent & Robotic Systems*, vol. 84, no. 1, pp. 371–386, 2016.
 - [3] V. Tereshchuk, J. Stewart, N. Bykov, S. Pedigo, S. Devasia, and A. G. Banerjee, “An efficient scheduling algorithm for multi-robot task allocation in assembling aircraft structures,” *IEEE Robotics and Automation Letters*, vol. 4, no. 4, pp. 3844–3851, 2019.
 - [4] K. Okumura and X. Défago, “Solving simultaneous target assignment and path planning efficiently with time-independent execution,” in *Proceedings of the International Conference on Automated Planning and Scheduling*, vol. 32, 2022, pp. 270–278.
 - [5] I. Draganjac, D. Miklič, Z. Kovačić, G. Vasiljević, and S. Bogdan, “Decentralized control of multi-agv systems in autonomous warehousing applications,” *IEEE Transactions on Automation Science and Engineering*, vol. 13, no. 4, pp. 1433–1447, 2016.
 - [6] H. Ma and S. Koenig, “Optimal target assignment and path finding for teams of agents,” *arXiv preprint arXiv:1612.05693*, 2016.
 - [7] Y. Tang, Z. Ren, J. Li, and K. Sycara, “Solving multi-agent target assignment and path finding with a single constraint tree,” in *2023 International Symposium on Multi-Robot and Multi-Agent Systems (MRS)*. IEEE, 2023, pp. 8–14.
 - [8] W. Hönl, S. Kiesel, A. Tinka, J. Durham, and N. Ayanian, “Conflict-based search with optimal task assignment,” in *Proceedings of the International Joint Conference on Autonomous Agents and Multiagent Systems*, 2018.
 - [9] G. Sharon, R. Stern, A. Felner, and N. R. Sturtevant, “Conflict-based search for optimal multi-agent pathfinding,” *Artificial intelligence*, vol. 219, pp. 40–66, 2015.
 - [10] R. Stern, N. Sturtevant, A. Felner, S. Koenig, H. Ma, T. Walker, J. Li, D. Atzmon, L. Cohen, T. Kumar *et al.*, “Multi-agent pathfinding: Definitions, variants, and benchmarks,” in *Proceedings of the International Symposium on Combinatorial Search*, vol. 10, no. 1, 2019, pp. 151–158.
 - [11] J. Yu and S. LaValle, “Structure and intractability of optimal multi-robot path planning on graphs,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 27, no. 1, 2013, pp. 1443–1449.
 - [12] P. Surynek, “An optimization variant of multi-robot path planning is intractable,” in *Proceedings of the AAAI conference on artificial intelligence*, vol. 24, no. 1, 2010, pp. 1261–1263.
 - [13] R. Luna and K. E. Bekris, “Push and swap: Fast cooperative pathfinding with completeness guarantees,” in *IJCAI*, vol. 11, 2011, pp. 294–300.
 - [14] B. De Wilde, A. W. Ter Mors, and C. Witteveen, “Push and rotate: a complete multi-agent pathfinding algorithm,” *Journal of Artificial Intelligence Research*, vol. 51, pp. 443–492, 2014.
 - [15] D. Silver, “Cooperative pathfinding,” in *Proceedings of the aai conference on artificial intelligence and interactive digital entertainment*, vol. 1, no. 1, 2005, pp. 117–122.
 - [16] J. Yu and S. M. LaValle, “Optimal multirobot path planning on graphs: Complete algorithms and effective heuristics,” *IEEE Transactions on Robotics*, vol. 32, no. 5, pp. 1163–1177, 2016.
 - [17] T. Standley, “Finding optimal solutions to cooperative pathfinding problems,” in *Proceedings of the AAAI conference on artificial intelligence*, vol. 24, no. 1, 2010, pp. 173–178.
 - [18] G. Wagner and H. Choset, “M*: A complete multirobot path planning algorithm with performance bounds,” in *2011 IEEE/RSJ international conference on intelligent robots and systems*. IEEE, 2011, pp. 3260–3267.
 - [19] J. Kottlinger, S. Almagor, and M. Lahijanian, “Conflict-based search for explainable multi-agent path finding,” in *Proceedings of the International Conference on Automated Planning and Scheduling*, vol. 32, 2022, pp. 692–700.
 - [20] A. Moldagaliyeva, J. Ortiz-Haro, M. Toussaint, and W. Hönl, “db-cbs: Discontinuity-bounded conflict-based search for multi-robot kinodynamic motion planning,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 14 569–14 575.
 - [21] H. W. Kuhn, “The hungarian method for the assignment problem,” *Naval research logistics quarterly*, vol. 2, no. 1-2, pp. 83–97, 1955.
 - [22] K. G. Murty, “An algorithm for ranking all the assignments in order of increasing cost,” *Operations research*, vol. 16, no. 3, pp. 682–687, 1968.
 - [23] L. R. Ford Jr and D. R. Fulkerson, “Maximal flow through a network,” *Canadian journal of Mathematics*, vol. 8, pp. 399–404, 1956.
 - [24] D. P. Bertsekas, “The auction algorithm: A distributed relaxation method for the assignment problem,” *Annals of operations research*, vol. 14, no. 1, pp. 105–123, 1988.
 - [25] G. Wagner, H. Choset, and N. Ayanian, “Subdimensional expansion and optimal task reassignment,” in *Proceedings of the International Symposium on Combinatorial Search*, vol. 3, no. 1, 2012, pp. 177–178.
 - [26] H. Ma, J. Li, T. Kumar, and S. Koenig, “Lifelong multi-agent path finding for online pickup and delivery tasks,” *arXiv preprint arXiv:1705.10868*, 2017.
 - [27] K. Okumura and X. Défago, “Solving simultaneous target assignment and path planning efficiently with time-independent execution,” *Artificial Intelligence*, vol. 321, p. 103946, 2023.
 - [28] P. Surynek, “An optimization variant of multi-robot path planning is intractable,” in *Proceedings of the AAAI conference on artificial intelligence*, vol. 24, no. 1, 2010, pp. 1261–1263.
 - [29] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
 - [30] A. Paradise, S. Surve, J. C. Menezes, M. Gupta, V. Bisht, K. R. Jang, C. Liu, S. Qiu, J. Dong, J. Shin, and S. Ferrari, “RealTHASC—a cyber-physical XR testbed for AI-supported real-time human autonomous systems collaborations,” *Front. Virtual Real.*, vol. 4, Dec. 2023.
 - [31] S. Surve, J. Guo, J. C. Menezes, C. Tate, Y. Jin, J. Walker, and S. Ferrari, “UnRealTHASC – a cyber-physical XR testbed for underwater real-time human autonomous systems collaboration,” in *2024 33rd IEEE International Conference on Robot and Human Interactive Communication (ROMAN)*. IEEE, Aug. 2024, pp. 196–203.